ADITYA UNIVERSITY

**SCHOOL OF COMPUTING**

**Department of Artificial Intelligence and Machine Learning**

# E-COMMERCE ORDER MANAGEMENT SYSTEM

*A Cornerstone Project Report*

***BATCH NUMBER : 11***

Submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

in

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**Submitted by**


| | |
|---|---|
| **P CHANDRIKA - AIML** | **(25B11AI911)** |
| **G ANVITHA - AIML** | **(25B11AI332)** |
| **J TANISHKA - AIML** | **(25B11AI420)** |
| **S SUDHADEVI - AIML** | **(25B11AIB23)** |


Under the guidance of

**Dr. P. Kishorekumar, PhD.**
Associate Professor, Department of AIML

**ADITYA UNIVERSITY, SURAMPALEM**

**Academic Year: 2026 - 2027**

ADITYA UNIVERSITY

**SCHOOL OF COMPUTING**

**Department of Artificial Intelligence and Machine Learning**

# CERTIFICATE

This is to certify that the Cornerstone project work entitled ***"E-COMMERCE ORDER MANAGEMENT SYSTEM"*** is submitted by

| | |
|---|---|
| **P CHANDRIKA - AIML** | **(25B11AI911)** |
| **G ANVITHA - AIML** | **(25B11AI332)** |
| **J TANISHKA - AIML** | **(25B11AI420)** |
| **S SUDHADEVI - AIML** | **(25B11AIB23)** |

In partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2026-2027.

**Project Guide**

**Dr. P. Kishorekumar, PhD.**

Associate Professor, Department of AIML

**Head of the Department:**

**Dr. Kovvuri N Bhargavi**

Associate Professor & HOD

# DECLARATION

We hereby declare that the Cornerstone project entitled **"E-COMMERCE ORDER MANAGEMENT SYSTEM"** is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. Degree in Artificial Intelligence and Machine Learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date:

**By**


| | |
|---|---|
| **P CHANDRIKA - AIML** | **(25B11AI911)** |
| **G ANVITHA - AIML** | **(25B11AI332)** |
| **J TANISHKA - AIML** | **(25B11AI420)** |
| **S SUDHADEVI - AIML** | **(25B11AIB23)** |

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. P. Kishorekumar, PhD. Associate Professor, Department of AIML,** who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. B. Manikyala Rao, Associate Professor**, **Dr. R. Suneetha, Assistant Professor and Dr. D. Veeraswamy, Assistant Professor** for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Kovvuri N Bhargavi,** Associate Professor & HOD**,** for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

**By**

| | |
|---|---|
| **P CHANDRIKA - AIML** | **(25B11AI911)** |
| **G ANVITHA - AIML** | **(25B11AI332)** |
| **J TANISHKA - AIML** | **(25B11AI420)** |
| **S SUDHADEVI - AIML** | **(25B11AIB23)** |

# TABLE OF CONTENTS


1. Cover Page....1
2. Certificate....2
3. Declaration....3
4. Acknowledgement....4
5. Abstract....6
6. List of Figures / List of Tables / Acronyms....7
7. Introduction....8
8. System Analysis....10
9. Database Design....13
10. ER Diagram....15
11. Relational Schema....16
12. Table Design....17
13. Normalization....20
14. SQL Implementation....22
15. SQL Queries & Screenshots....25
16. Conclusion & Future Enhancements....44
17. References....45

# ABSTRACT

The E-Commerce Order Management System is a relational database designed to organise and manage the information handled by an online shopping platform every day. An online store must keep track of its customers, the products it sells, the categories those products belong to, the orders customers place, the items inside each order, the payments received, and the delivery of every order. When these records are kept manually or in disconnected spreadsheets, they lead to duplicate entries, inconsistent order information, and difficulty in tracking the status of an order. The proposed system models these entities and the relationships between them using a relational database approach and stores the information in a centralized database that enforces data integrity through Primary Keys, Foreign Keys, and constraints.

The database has been designed with seven entities — Customer, Category, Product, Order, Order_Item, Payment, and Delivery — connected through well-defined one-to-many relationships. The design has been normalized up to the Third Normal Form (3NF) to minimise redundancy and prevent update anomalies. The system has been implemented in MySQL using Data Definition Language (DDL) for table creation, Data Manipulation Language (DML) for sample records and CRUD operations, and Data Query Language (DQL) for filtering, sorting, joins, aggregate functions, grouping, and a database view. The resulting design provides a structured foundation for customer management, product and category management, order processing, order-status tracking, payment management, and delivery management.

## LIST OF FIGURES

Figure 1. ER Diagram of the E-Commerce Order Management System

## LIST OF TABLES

Table 1. Major Database Entities

Table 2. Entity Relationships and Cardinality

Table 3. Relational Schema Summary

Table 4. Customer Table Structure

Table 5. Category Table Structure

Table 6. Product Table Structure

Table 7. Order Table Structure

Table 8. Order_Item Table Structure

Table 9. Payment Table Structure

Table 10. Delivery Table Structure

Table 11. Keys and Constraints Summary

## ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- CRUD – Create, Read, Update, Delete
- PK – Primary Key
- FK – Foreign Key
- UNF – Unnormalized Form
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form
- UPI – Unified Payments Interface

# 1. INTRODUCTION

## 1.1 Brief Information about the Project

Online shopping platforms handle large amounts of interconnected information related to customers, products, product categories, orders, payments, and deliveries. A database provides a structured and efficient way to organise and manage these records. The proposed E-Commerce Order Management System is designed to manage online customer orders efficiently. It helps customers place orders for products through an online platform, stores customer, product, order, and payment details, and allows the administrator to add, update, and manage products. The system tracks the status of every order — Pending, Confirmed, Shipped, Delivered, or Cancelled — and thereby reduces manual work and improves order management.

The project focuses on designing the database structure — including entities, attributes, Primary Keys, Foreign Keys, relationships, and the ER diagram — and implementing it through SQL. The project domain is E-Commerce / Online Shopping, and the technologies used are DBMS and SQL (MySQL).

## 1.2 Motivation and Contribution of the Project

The project is motivated by the practical difficulties faced when customer, product, and order records are maintained manually or in separate files. Without a centralized system, data is duplicated, order information becomes inconsistent, order status is hard to trace, and generating reports is time-consuming. A centralized relational database provides a systematic approach to storing related e-commerce data and to enforcing the relationships between them through keys and constraints. The contributions of the project are:

- Centralizing customer, category, product, order, payment, and delivery records in one relational database.
- Reducing duplicate and inconsistent order information through keys and constraints.
- Maintaining referential integrity using Primary and Foreign Keys.
- Supporting quick retrieval of order, payment, and delivery information through SQL queries and joins.
- Providing reports and summaries through aggregate functions, grouping, and views.

## 1.3 Objectives of the Project

- Identify the major entities involved in an e-commerce order management system.
- Identify suitable attributes for each entity.
- Establish relationships between the identified entities.
- Design an Entity–Relationship (ER) diagram.
- Convert the ER model into a relational database schema.
- Identify suitable Primary Keys and Foreign Keys.
- Normalize the database up to the Third Normal Form (3NF).
- Implement the design in SQL and demonstrate DDL, DML, DQL, joins, aggregate functions, CRUD operations, and views.

## 1.4 Scope of the Project

The scope of the project covers the database-oriented management of core e-commerce information. The system connects customer details and contact information (Customer), product categories (Category), product name, price, stock, and category (Product), order date, total amount, and order status (Order), the products and quantities contained in each order (Order_Item), payment method, date, and status (Payment), and delivery address, date, and status (Delivery). The database can serve as the backend for an online shopping application and can be extended with modules such as role-based access, inventory management, online payment gateway integration, and real-time order tracking.

# 2. SYSTEM ANALYSIS

## 2.1 Existing System

- Manual maintenance of order records.
- Customer and product data stored separately in disconnected files.
- Duplicate or inconsistent order information.
- Difficulty in tracking the status of an order.
- Time-consuming searches for a particular order.
- Difficulty in generating consolidated reports.

## 2.2 Proposed System

The proposed system uses a centralized relational database that connects e-commerce records through well-defined entities, relationships, Primary Keys, and Foreign Keys. Keys and constraints (NOT NULL, UNIQUE, and CHECK) maintain data integrity, and SQL queries provide quick retrieval of orders and reports.

- Centralized database for customers, categories, products, orders, order items, payments, and deliveries.
- Related tables for Customers, Products, Orders, and Payments.
- Order-status tracking: Pending, Confirmed, Shipped, Delivered, and Cancelled.
- SQL queries for quick order retrieval.
- SQL joins, aggregation, and views for reports.
- Referential integrity across all related tables.

## 2.3 Functional Requirements

| Module | Functional Requirement |
|---|---|
| Customer Management | Manage customer details, contact information, and account records. |
| Product Management | Manage product details, categories, prices, and stock information. |
| Order Management | Manage order dates, quantities, total amounts, and order status. |
| Payment Management | Manage payment methods, payment dates, and payment status. |
| Delivery Management | Manage delivery address, delivery date, and delivery status. |
| Order Tracking | Track the progress of an order from placement to final delivery. |
| Database Management | Maintain relationships between customers, products, orders, payments, and deliveries using primary and foreign keys. |

## 2.4 Non-Functional Requirements

- **Performance** – queries on customer, product, and order data should return results efficiently.
- **Integrity** – invalid references (for example, a product in a category that does not exist) and duplicate key values must be prevented.
- **Security** – access to customer and payment information should be controlled for authorized users.
- **Usability** – the database structure should be understandable and easy for administrators to use.
- **Reliability** – operations such as placing an order and recording a payment should preserve a consistent database state.
- **Scalability** – the design should support growth in customers, products, and orders over time.

## 2.5 Requirements Specification

| Category | Requirement |
|---|---|
| Hardware | Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse. |
| Operating System | Windows 10/11 or an equivalent supported platform. |
| Database | MySQL Relational Database Management System (MySQL Workbench as the client). |

# 3. DATABASE DESIGN

## 3.1 Major Entities

Seven major entities have been identified for the proposed E-Commerce Order Management System, as summarised in Table 1.

**Table 1:** Major Database Entities

| Entity | Description |
|---|---|
| CUSTOMER | Stores customer name, email, phone, and address. |
| CATEGORY | Stores product category name and description. |
| PRODUCT | Stores product name, price, stock quantity, and its category. |
| ORDER | Stores the order date, total amount, order status, and the customer who placed it. |
| ORDER_ITEM | Stores each product entry in an order along with quantity and subtotal. |
| PAYMENT | Stores the payment method, payment date, and payment status of an order. |
| DELIVERY | Stores the delivery address, delivery date, and delivery status of an order. |

## 3.2 ER Model

The ER model represents the major entities of the e-commerce system and the relationships between them. CUSTOMER is the parent of ORDER: a customer may place many orders, while every order belongs to exactly one customer. CATEGORY is the parent of PRODUCT: a category may contain many products. ORDER_ITEM links an ORDER to the PRODUCTs it contains; because an order may contain many products and a product may appear in many orders, ORDER_ITEM resolves this many-to-many association into two one-to-many relationships. PAYMENT and DELIVERY each reference the ORDER through the Order_ID foreign key, so that the payment and delivery details of every order are stored in their own tables.

## 3.3 Relationships and Cardinality

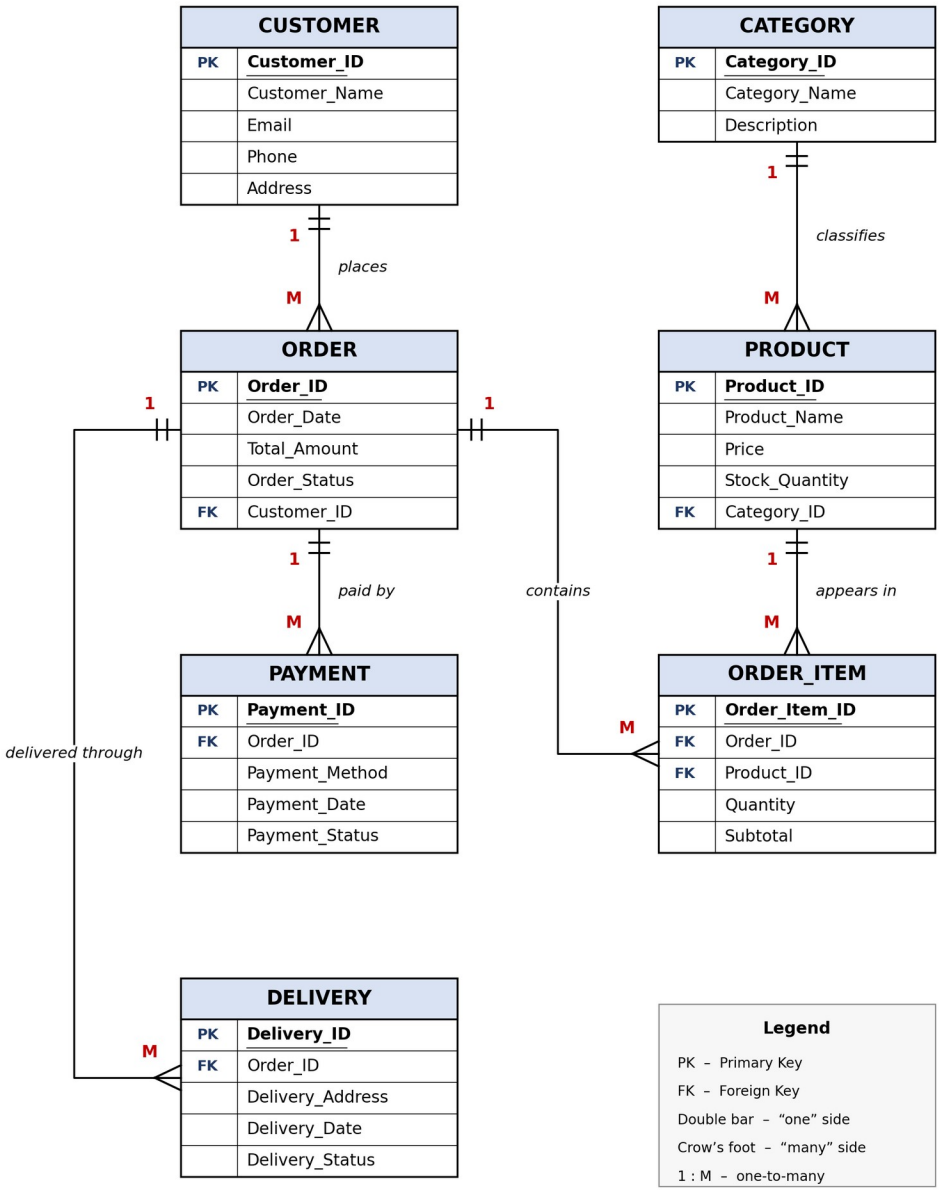
**Table 2:** Entity Relationships and Cardinality

| Relationship | Cardinality | Meaning |
|---|---|---|
| CUSTOMER – ORDER | 1 : M | One customer can place many orders. |
| CATEGORY – PRODUCT | 1 : M | One category can contain many products. |
| ORDER – ORDER_ITEM | 1 : M | One order can contain many order items. |
| PRODUCT – ORDER_ITEM | 1 : M | One product can appear in many order items. |
| ORDER – PAYMENT | 1 : M | An order is linked to its payment record(s) through Order_ID. |
| ORDER – DELIVERY | 1 : M | An order is linked to its delivery record(s) through Order_ID. |

# 4. ER DIAGRAM

The ER diagram shows the seven entities of the E-Commerce Order Management System and the one-to-many relationships between them. Primary Keys (PK) and Foreign Keys (FK) are marked within each entity box, consistent with the relational schema and the SQL implementation presented in the following sections. The crow's-foot symbol marks the "many" side of a relationship and the double bar marks the "one" side.

**Figure 1: ER Diagram of the E-Commerce Order Management System**



## 5. RELATIONAL SCHEMA

The relational schema below follows the table-creation order used in the SQL implementation, so that every Foreign Key already has a valid parent table defined before it.

**Table 3:** Relational Schema Summary

| Relation | Relational Schema | Keys |
|---|---|---|
| CUSTOMER | Customer(Customer_ID, Customer_Name, Email, Phone, Address) | PK: Customer_ID |
| CATEGORY | Category(Category_ID, Category_Name, Description) | PK: Category_ID |
| PRODUCT | Product(Product_ID, Product_Name, Price, Stock_Quantity, Category_ID) | PK: Product_ID; FK: Category_ID |
| ORDER | Order(Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) | PK: Order_ID; FK: Customer_ID |
| ORDER_ITEM | Order_Item(Order_Item_ID, Order_ID, Product_ID, Quantity, Subtotal) | PK: Order_Item_ID; FK: Order_ID, Product_ID |
| PAYMENT | Payment(Payment_ID, Order_ID, Payment_Method, Payment_Date, Payment_Status) | PK: Payment_ID; FK: Order_ID |
| DELIVERY | Delivery(Delivery_ID, Order_ID, Delivery_Address, Delivery_Date, Delivery_Status) | PK: Delivery_ID; FK: Order_ID |

Product references Category; Order references Customer; Order_Item references Order and Product; and Payment and Delivery each reference Order. Since ORDER is a reserved word in SQL, the table name is written with back-quotes (`ORDER`) in all queries.

# 6. TABLE DESIGN

## 6.1 Customer Table

**Table 4:** Customer Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Customer_ID | INT | PK | Unique customer identifier |
| Customer_Name | VARCHAR(100) | NOT NULL | Customer's full name |
| Email | VARCHAR(100) | UNIQUE | Customer's email address |
| Phone | VARCHAR(15) | — | Customer's contact number |
| Address | VARCHAR(200) | — | Customer's address |

## 6.2 Category Table

**Table 5:** Category Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Category_ID | INT | PK | Unique category identifier |
| Category_Name | VARCHAR(100) | NOT NULL | Name of the category |
| Description | VARCHAR(200) | — | Short description of the category |

## 6.3 Product Table

**Table 6:** Product Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Product_ID | INT | PK | Unique product identifier |
| Product_Name | VARCHAR(100) | NOT NULL | Name of the product |
| Price | DECIMAL(10,2) | CHECK (Price > 0) | Unit price of the product |
| Stock_Quantity | INT | CHECK (Stock_Quantity >= 0) | Quantity available in stock |
| Category_ID | INT | FK | References Category |

## 6.4 Order Table

**Table 7:** Order Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Order_ID | INT | PK | Unique order identifier |
| Customer_ID | INT | FK | References Customer |
| Order_Date | DATE | — | Date on which the order was placed |
| Total_Amount | DECIMAL(10,2) | CHECK (Total_Amount >= 0) | Total amount of the order |
| Order_Status | VARCHAR(30) | — | Pending, Confirmed, Shipped, Delivered, or Cancelled |

## 6.5 Order_Item Table

**Table 8:** Order_Item Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Order_Item_ID | INT | PK | Unique order-item identifier |
| Order_ID | INT | FK | References Order |
| Product_ID | INT | FK | References Product |
| Quantity | INT | CHECK (Quantity > 0) | Quantity of the product ordered |
| Subtotal | DECIMAL(10,2) | CHECK (Subtotal >= 0) | Amount for this order item |

## 6.6 Payment Table

**Table 9:** Payment Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Payment_ID | INT | PK | Unique payment identifier |
| Order_ID | INT | FK | References Order |
| Payment_Method | VARCHAR(30) | — | UPI, Card, Cash, etc. |
| Payment_Date | DATE | — | Date of payment |
| Payment_Status | VARCHAR(30) | — | Paid or Pending |

## 6.7 Delivery Table

**Table 10:** Delivery Table Structure

| Attribute | Data Type | Constraint | Description |
|---|---|---|---|
| Delivery_ID | INT | PK | Unique delivery identifier |
| Order_ID | INT | FK | References Order |
| Delivery_Address | VARCHAR(200) | — | Address to which the order is delivered |
| Delivery_Date | DATE | — | Expected or actual delivery date |
| Delivery_Status | VARCHAR(30) | — | Pending, Confirmed, Shipped, or Delivered |

## 6.8 Keys and Constraints

**Table 11:** Keys and Constraints Summary

| Constraint Type | Applied To / Purpose |
|---|---|
| Primary Key | Customer_ID, Category_ID, Product_ID, Order_ID, Order_Item_ID, Payment_ID, Delivery_ID |
| Foreign Key | Product.Category_ID; Order.Customer_ID; Order_Item.Order_ID, Order_Item.Product_ID; Payment.Order_ID; Delivery.Order_ID |
| NOT NULL | Customer.Customer_Name, Category.Category_Name, Product.Product_Name |
| UNIQUE | Customer.Email |
| CHECK | Product.Price (> 0), Product.Stock_Quantity (>= 0), Order.Total_Amount (>= 0), Order_Item.Quantity (> 0), Order_Item.Subtotal (>= 0) |
| Referential Integrity | Every Category_ID in Product, Customer_ID in Order, Order_ID and Product_ID in Order_Item, and Order_ID in Payment and Delivery must exist in the parent table. |

# 7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The following example uses a single, unnormalized order record — covering the customer, the products ordered, the payment, and the delivery — to illustrate the process from the Unnormalized Form (UNF) to the Third Normal Form (3NF).

## 7.1 Unnormalized Form (UNF)

A single order record might contain Order_ID, Order_Date, Order_Status, Customer_Name, Email, Phone, Address, together with several product entries (Product_Name, Price, Category_Name, Quantity) and the payment and delivery details of the order, all stored in one record. Repeating product groups and repeated customer, category, payment, and delivery details create redundancy.

## 7.2 First Normal Form (1NF)

In 1NF, repeating groups and multi-valued attributes are removed and every attribute holds a single (atomic) value. Each record is uniquely identified using a Primary Key. For example, ORDER_ITEM stores one product entry per order item, so an order containing several products is represented by several rows instead of a repeating group.

## 7.3 Second Normal Form (2NF)

The database is already in 1NF. In 2NF, non-key attributes must depend on the whole primary key. Product details (name, price, stock) depend only on Product_ID and are separated into PRODUCT; customer details (name, email, phone, address) depend only on Customer_ID and are separated into CUSTOMER; and order details (date, total amount, status) depend only on Order_ID and are separated into ORDER.

## 7.4 Third Normal Form (3NF)

The database is already in 2NF. In 3NF, non-key attributes must depend only on the primary key and not on other non-key attributes. Category information (name, description) is stored once in CATEGORY instead of being repeated for every product; payment information is stored in PAYMENT; and delivery information is stored in DELIVERY. This reduces data redundancy and improves data consistency.

**Final result:** Relations in 3NF

| Relation | Reason for Separation |
|---|---|
| Customer | Stores customer-specific attributes once, independent of any order. |
| Category | Stores category details once, instead of repeating them for every product. |
| Product | Stores product-specific attributes and a Category_ID reference. |
| Order | Stores order-specific attributes and a Customer_ID reference. |
| Order_Item | Stores one product entry per order item with its quantity and subtotal. |
| Payment | Stores payment details linked to an order. |
| Delivery | Stores delivery details linked to an order. |

The proposed database is therefore normalized up to 3NF by separating customer, category, product, order, order-item, payment, and delivery information into related tables using primary and foreign keys.

# 8. SQL IMPLEMENTATION

The SQL implementation was executed in MySQL. Tables are created in an order that satisfies every Foreign Key dependency: CUSTOMER and CATEGORY first (no dependencies), then PRODUCT (depends on CATEGORY) and ORDER (depends on CUSTOMER), then ORDER_ITEM (depends on ORDER and PRODUCT), and finally PAYMENT and DELIVERY (depend on ORDER).

## 8.1 DDL – Data Definition Language

DDL commands used: CREATE TABLE creates the database tables; PRIMARY KEY uniquely identifies each record; FOREIGN KEY establishes relationships between tables; NOT NULL ensures required fields contain values; UNIQUE prevents duplicate values; and CHECK validates values based on conditions.

```
SHOW DATABASES;
USE COLLEGEDB;

-- 1. CUSTOMER
CREATE TABLE CUSTOMER (
    CUSTOMER_ID INT PRIMARY KEY,
    CUSTOMER_NAME VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(200)
);

-- 2. CATEGORY
CREATE TABLE CATEGORY (
    CATEGORY_ID INT PRIMARY KEY,
    CATEGORY_NAME VARCHAR(100) NOT NULL,
    Description VARCHAR(200)
);

-- 3. PRODUCT (references CATEGORY)
CREATE TABLE PRODUCT (
    PRODUCT_ID INT PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    STOCK_QUANTITY INT CHECK (STOCK_QUANTITY >= 0),
    CATEGORY_ID INT,
    FOREIGN KEY (CATEGORY_ID) REFERENCES CATEGORY(CATEGORY_ID)
);

-- 4. ORDER (references CUSTOMER)
CREATE TABLE `ORDER` (
    ORDER_ID INT PRIMARY KEY,
    CUSTOMER_ID INT,
```

```
    ORDER_DATE DATE,
    TOTAL_AMOUNT DECIMAL(10,2) CHECK (TOTAL_AMOUNT >= 0),
    ORDER_STATUS VARCHAR(30),
    FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
);

-- 5. ORDER_ITEM (references ORDER, PRODUCT)
CREATE TABLE ORDER_ITEM (
    ORDER_ITEM_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PRODUCT_ID INT,
    Quantity INT CHECK (Quantity > 0),
    Subtotal DECIMAL(10,2) CHECK (Subtotal >= 0),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID),
    FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID)
);

-- 6. PAYMENT (references ORDER)
CREATE TABLE PAYMENT (
    PAYMENT_ID INT PRIMARY KEY,
    ORDER_ID INT,
    PAYMENT_METHOD VARCHAR(30),
    PAYMENT_DATE DATE,
    PAYMENT_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID)
);

-- 7. DELIVERY (references ORDER)
CREATE TABLE DELIVERY (
    DELIVERY_ID INT PRIMARY KEY,
    ORDER_ID INT,
    DELIVERY_ADDRESS VARCHAR(200),
    DELIVERY_DATE DATE,
    DELIVERY_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID) REFERENCES `ORDER`(ORDER_ID)
);
```

## 8.2 DML – Data Manipulation Language

DML commands used: INSERT adds customer, category, product, order, order-item, payment, and delivery records; UPDATE modifies product stock, order status, payment status, or delivery status; DELETE removes records when required; and SELECT retrieves information. Each table is populated with valid, foreign-key-consistent sample records.

```
-- CUSTOMER
INSERT INTO CUSTOMER VALUES
(1, 'Ravi',  'ravi@gmail.com',  '9876543210', 'Hyderabad'),
(2, 'Anu',   'anu@gmail.com',   '9876543211', 'Vijayawada'),
(3, 'Kiran', 'kiran@gmail.com', '9876543212', 'Rajahmundry'),
(4, 'Sneha', 'sneha@gmail.com', '9876543213', 'Visakhapatnam');

-- CATEGORY
```

```
INSERT INTO CATEGORY VALUES
(101, 'Electronics', 'Electronic Products'),
(102, 'Clothing',    'Fashion Products'),
(103, 'Books',       'Educational Books');

-- PRODUCT
INSERT INTO PRODUCT VALUES
(201, 'Headphones',  1500.00, 20, 101),
(202, 'Smart Watch', 2500.00, 15, 101),
(203, 'T-Shirt',      800.00, 30, 102),
(204, 'DBMS Book',    600.00, 25, 103);

-- ORDER
INSERT INTO `ORDER` VALUES
(301, 1, '2026-09-28', 1500.00, 'Shipped'),
(302, 2, '2026-09-28', 2500.00, 'Pending'),
(303, 3, '2026-09-29',  800.00, 'Delivered'),
(304, 4, '2026-09-29',  600.00, 'Confirmed');

-- ORDER_ITEM
INSERT INTO ORDER_ITEM VALUES
(401, 301, 201, 1, 1500.00),
(402, 302, 202, 1, 2500.00),
(403, 303, 203, 1,  800.00),
(404, 304, 204, 1,  600.00);

-- PAYMENT
INSERT INTO PAYMENT VALUES
(501, 301, 'UPI',  '2026-09-28', 'Paid'),
(502, 302, 'Card', '2026-09-28', 'Pending'),
(503, 303, 'Cash', '2026-09-29', 'Paid'),
(504, 304, 'UPI',  '2026-09-29', 'Paid');

-- DELIVERY
INSERT INTO DELIVERY VALUES
(601, 301, 'Hyderabad',     '2026-09-30', 'Shipped'),
(602, 302, 'Vijayawada',    '2026-10-01', 'Pending'),
(603, 303, 'Rajahmundry',   '2026-09-30', 'Delivered'),
(604, 304, 'Visakhapatnam', '2026-10-02', 'Confirmed');
```

## 8.3 DQL – Data Query Language

DQL is used to retrieve data from the tables: records are filtered using WHERE, sorted using ORDER BY, and refined using DISTINCT, LIKE, and IS NULL / IS NOT NULL; only the required columns are retrieved; and JOIN is used to retrieve data from related tables. The full set of queries with their outputs is presented in Section 9.

# 9. SQL QUERIES & SCREENSHOTS

The queries below are grouped by category — table creation and sample data, filtering and sorting, joins, aggregate functions with grouping, CRUD operations, a view, and data-integrity validation. A screenshot of the actual MySQL output is provided with each query.

## 9.1 Table Creation Output (DDL)

**Q1. Create the CUSTOMER table (CREATE TABLE)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  CREATE TABLE CUSTOMERS (
4      CUSTOMERS_ID INT PRIMARY KEY,
5      CUSTOMERs_NAME VARCHAR(100) NOT NULL,
6      Email VARCHAR(100) UNIQUE,
7      Phone VARCHAR(15),
8      Address VARCHAR(200)
9  );
```

**Q2. Create the CATEGORY table (CREATE TABLE)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  CREATE TABLE CATEGORY (
4      CATEGORY_ID INT PRIMARY KEY,
5      CATEGORY_NAME VARCHAR(100) NOT NULL,
6      Description VARCHAR(200)
7  );
```

**Q3. Create the PRODUCT table (CREATE TABLE)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE PRODUCT (
    PRODUCT_ID INT PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    STOCK_QUANTITY INT CHECK (STOCK_QUANTITY >= 0),
    CATEGORY_ID INT,
    FOREIGN KEY (CATEGORY_ID)
        REFERENCES CATEGORY(CATEGORY_ID)
);
```

**Q4. Create the ORDER table (CREATE TABLE)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE `ORDER` (
    ORDER_ID INT PRIMARY KEY,
    CUSTOMERS_ID INT,
    ORDER_DATE DATE,
    TOTAL_AMOUNT DECIMAL(10,2) CHECK (TOTAL_AMOUNT >= 0),
    ORDER_STATUS VARCHAR(30),
    FOREIGN KEY (CUSTOMERS_ID)
        REFERENCES CUSTOMERS(CUSTOMERS_ID)
);
```

**Q5. Create the ORDER_ITEM table (CREATE TABLE)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  CREATE TABLE ORDER_ITEM (
4      ORDER_ITEM_ID INT PRIMARY KEY,
5      ORDER_ID INT,
6      PRODUCT_ID INT,
7      Quantity INT CHECK (Quantity > 0),
8      Subtotal DECIMAL(10,2) CHECK (Subtotal >= 0),
9      FOREIGN KEY (ORDER_ID)
10         REFERENCES `ORDER`(ORDER_ID),
11     FOREIGN KEY (PRODUCT_ID)
12         REFERENCES PRODUCT(PRODUCT_ID)
13 );
```

**Q6. Create the PAYMENT table (CREATE TABLE)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  CREATE TABLE PAYMENT (
4      PAYMENT_ID INT PRIMARY KEY,
5      ORDER_ID INT,
6      PAYMENT_METHOD VARCHAR(30),
7      PAYMENT_DATE DATE,
8      PAYMENT_STATUS VARCHAR(30),
9      FOREIGN KEY (ORDER_ID)
10         REFERENCES `ORDER`(ORDER_ID)
11 );
```

**Q7. Create the DELIVERY table (CREATE TABLE)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
CREATE TABLE DELIVERY (
    DELIVERY_ID INT PRIMARY KEY,
    ORDER_ID INT,
    DELIVERY_ADDRESS VARCHAR(200),
    DELIVERY_DATE DATE,
    DELIVERY_STATUS VARCHAR(30),
    FOREIGN KEY (ORDER_ID)
        REFERENCES `ORDER`(ORDER_ID)
);
```

## 9.2 Inserting Sample Records (DML)

**Q8. Insert records into the CUSTOMER table (INSERT)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CUSTOMERS
VALUES
(1, 'Ravi', 'ravi@gmail.com', '9876543210', 'Hyderabad'),
(2, 'Anu', 'anu@gmail.com', '9876543211', 'Vijayawada'),
(3, 'Kiran', 'kiran@gmail.com', '9876543212', 'Rajahmundry'),
(4, 'Sneha', 'sneha@gmail.com', '9876543213', 'Visakhapatnam');
```

**Q9. Insert records into the CATEGORY table (INSERT)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO CATEGORY
VALUES
(101, 'Electronics', 'Electronic Products'),
(102, 'Clothing', 'Fashion Products'),
(103, 'Books', 'Educational Books');
```

**Q10. Insert records into the PRODUCT table (INSERT)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO PRODUCT
VALUES
(201, 'Headphones', 1500.00, 20, 101),
(202, 'Smart Watch', 2500.00, 15, 101),
(203, 'T-Shirt', 800.00, 30, 102),
(204, 'DBMS Book', 600.00, 25, 103);
```

**Q11. Insert records into the ORDER table (INSERT)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO `ORDER`
VALUES
(301, 1, '2026-09-28', 1500.00, 'Shipped'),
(302, 2, '2026-09-28', 2500.00, 'Pending'),
(303, 3, '2026-09-29', 800.00, 'Delivered'),
(304, 4, '2026-09-29', 600.00, 'Confirmed');
```

**Q12. Insert records into the ORDER_ITEM table (INSERT)**

```sql
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO ORDER_ITEM
VALUES
(401, 301, 201, 1, 1500.00),
(402, 302, 202, 1, 2500.00),
(403, 303, 203, 1, 800.00),
(404, 304, 204, 1, 600.00);
```

**Q13. Insert records into the PAYMENT table (INSERT)**

```
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO PAYMENT
VALUES
(501, 301, 'UPI', '2026-09-28', 'Paid'),
(502, 302, 'Card', '2026-09-28', 'Pending'),
(503, 303, 'Cash', '2026-09-29', 'Paid'),
(504, 304, 'UPI', '2026-09-29', 'Paid');
```

**Q14. Insert records into the DELIVERY table (INSERT)**

```
SHOW DATABASES;
USE COLLEGEDB;
INSERT INTO DELIVERY
VALUES
(601, 301, 'Hyderabad', '2026-09-30', 'Shipped'),
(602, 302, 'Vijayawada', '2026-10-01', 'Pending'),
(603, 303, 'Rajahmundry', '2026-09-30', 'Delivered'),
(604, 304, 'Visakhapatnam', '2026-10-02', 'Confirmed');
```

## 9.3 Basic SELECT, Filtering and Sorting

**Q16. Display the records of all seven tables (SELECT *)**

```
SHOW DATABASES;
USE COLLEGEDB;
SELECT * FROM CUSTOMER;
SELECT * FROM CATEGORY;
SELECT * FROM PRODUCT;
SELECT * FROM `ORDER`;
SELECT * FROM ORDER_ITEM;
SELECT * FROM PAYMENT;
SELECT * FROM DELIVERY;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| DELIVERY_ID | ORDER_ID | DELIVERY_ADDRESS | DELIVERY_DATE | DELIVERY_STATUS |
|---|---|---|---|---|
| 601 | 301 | Hyderabad | 2026-09-30 | Shipped |
| 602 | 302 | Vijayawada | 2026-10-01 | Pending |
| 603 | 303 | Rajahmundry | 2026-09-30 | Delivered |
| 604 | 304 | Visakhapatnam | 2026-10-02 | Confirmed |
| NULL | NULL | NULL | NULL | NULL |

**Q17. Products priced above 1000 (WHERE)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT PRODUCT_NAME, Price
4  FROM PRODUCT
5  WHERE Price > 1000;
```

Result Grid | Filter Rows:

| PRODUCT_NAME | Price |
|---|---|
| Headphones | 1500.00 |
| Smart Watch | 2500.00 |

**Q18. Products sorted by price, highest first (ORDER BY)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT *
4  FROM PRODUCT
5  ORDER BY Price DESC;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

| PRODUCT_ID | PRODUCT_NAME | Price | STOCK_QUANTITY | CATEGORY_ID |
|---|---|---|---|---|
| 202 | Smart Watch | 2500.00 | 15 | 101 |
| 201 | Headphones | 1500.00 | 20 | 101 |
| 203 | T-Shirt | 800.00 | 30 | 102 |
| 204 | DBMS Book | 600.00 | 25 | 103 |
| NULL | NULL | NULL | NULL | NULL |

**Q19. Distinct order statuses (DISTINCT)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT DISTINCT ORDER_STATUS
4    FROM `ORDER`;
```

Result Grid | Filter Rows:

| ORDER_STATUS |
|---|
| Shipped |
| Pending |
| Delivered |
| Confirmed |

**Q20. Customers whose name starts with 'A' (LIKE)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT *
4    FROM CUSTOMERS
5    WHERE CUSTOMERS_NAME LIKE 'A%';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| CUSTOMERS_ID | CUSTOMERs_NAME | Email | Phone | Address |
|---|---|---|---|---|
| 2 | Anu | anu@gmail.com | 9876543211 | Vijayawada |
| NULL | NULL | NULL | NULL | NULL |

**Q21. Customers who have an email address (IS NOT NULL)**

```
SHOW DATABASES;
USE COLLEGEDB;
SELECT *
FROM CUSTOMERS
WHERE Email IS NOT NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

| CUSTOMERS_ID | CUSTOMERs_NAME | Email | Phone | Address |
|---|---|---|---|---|
| 2 | Anu | anu@gmail.com | 9876543211 | Vijayawada |
| 3 | Kiran | kiran@gmail.com | 9876543212 | Rajahmundry |
| 1 | Ravi | ravi@gmail.com | 9876543210 | Hyderabad |
| 4 | Sneha | sneha@gmail.com | 9876543213 | Visakhapatnam |
| NULL | NULL | NULL | NULL | NULL |

## 9.4 Joins

**Q22. Customer name with their order details (INNER JOIN – CUSTOMER and ORDER)**

```
SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE,
       O.TOTAL_AMOUNT, O.ORDER_STATUS
FROM CUSTOMER C
INNER JOIN `ORDER` O
ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

***Output:***

| CUSTOMER_NAME | ORDER_ID | ORDER_DATE | TOTAL_AMOUNT | ORDER_STATUS |
|---|---|---|---|---|
| Ravi | 301 | 2026-09-28 | 1500.00 | Shipped |
| Anu | 302 | 2026-09-28 | 2500.00 | Pending |
| Kiran | 303 | 2026-09-29 | 800.00 | Delivered |
| Sneha | 304 | 2026-09-29 | 600.00 | Confirmed |

**Q23. Product with its category (INNER JOIN – PRODUCT and CATEGORY)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      P.PRODUCT_NAME,
5      P.Price,
6      C.CATEGORY_NAME
7  FROM PRODUCT P
8  INNER JOIN CATEGORY C
9  ON P.CATEGORY_ID = C.CATEGORY_ID;
```

Result Grid | Filter Rows:

| PRODUCT_NAME | Price | CATEGORY_NAME |
|---|---|---|
| Headphones | 1500.00 | Electronics |
| Smart Watch | 2500.00 | Electronics |
| T-Shirt | 800.00 | Clothing |
| DBMS Book | 600.00 | Books |

**Q24. Products in each order (INNER JOIN – ORDER, ORDER_ITEM and PRODUCT)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      O.ORDER_ID,
5      P.PRODUCT_NAME,
6      OI.Quantity,
7      OI.Subtotal
8  FROM `ORDER` O
9  INNER JOIN ORDER_ITEM OI
10 ON O.ORDER_ID = OI.ORDER_ID
11 INNER JOIN PRODUCT P
12 ON OI.PRODUCT_ID = P.PRODUCT_ID;
```

Result Grid | Filter Rows: | Exp

| ORDER_ID | PRODUCT_NAME | Quantity | Subtotal |
|---|---|---|---|
| 301 | Headphones | 1 | 1500.00 |
| 302 | Smart Watch | 1 | 2500.00 |
| 303 | T-Shirt | 1 | 800.00 |
| 304 | DBMS Book | 1 | 600.00 |

**Q25. Order status with payment and delivery status (INNER JOIN – ORDER, PAYMENT and DELIVERY)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT
4      O.ORDER_ID,
5      O.ORDER_STATUS,
6      P.PAYMENT_METHOD,
7      P.PAYMENT_STATUS,
8      D.DELIVERY_STATUS
9  FROM `ORDER` O
10 INNER JOIN PAYMENT P
11 ON O.ORDER_ID = P.ORDER_ID
12 INNER JOIN DELIVERY D
13 ON O.ORDER_ID = D.ORDER_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| ORDER_ID | ORDER_STATUS | PAYMENT_METHOD | PAYMENT_STATUS | DELIVERY_STATUS |
|---|---|---|---|---|
| 301 | Shipped | UPI | Paid | Shipped |
| 302 | Pending | Card | Pending | Pending |
| 303 | Delivered | Cash | Paid | Delivered |
| 304 | Confirmed | UPI | Paid | Confirmed |

## 9.5 Aggregate Functions and Grouping

**Q26. Total number of orders (COUNT)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT COUNT(*) AS TOTAL_ORDERS
4  FROM `ORDER`;
```

Result Grid | Filter Rows:

| TOTAL_ORDERS |
|---|
| 4 |

**Q27. Total sales amount (SUM)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

| TOTAL_SALES |
|---|
| 5400.00 |

**Q28. Average order amount (AVG)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_AMOUNT
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

| AVERAGE_ORDER_AMOUNT |
|---|
| 1350.000000 |

**Q29. Highest order amount (MAX)**

```
1  SHOW DATABASES;
2  USE COLLEGEDB;
3  SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER
4  FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

| HIGHEST_ORDER |
|---|
| 2500.00 |

**Q30. Lowest order amount (MIN)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDER
4    FROM `ORDER`;
```

Result Grid | Filter Rows: | Export:

| LOWEST_ORDER |
|---|
| 600.00 |

**Q31. Number of orders in each status (GROUP BY)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
4    FROM `ORDER`
5    GROUP BY ORDER_STATUS;
```

Result Grid | Filter Rows: | Export: | Wr

| ORDER_STATUS | TOTAL_ORDERS |
|---|---|
| Shipped | 1 |
| Pending | 1 |
| Delivered | 1 |
| Confirmed | 1 |

**Q32. Filtering grouped results (GROUP BY with HAVING)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT ORDER_STATUS, COUNT(*) AS TOTAL_ORDERS
4    FROM `ORDER`
5    GROUP BY ORDER_STATUS
6    HAVING COUNT(*) >= 1;
```

Result Grid | Filter Rows: | Export: | Wra

| ORDER_STATUS | TOTAL_ORDERS |
|---|---|
| Shipped | 1 |
| Pending | 1 |
| Delivered | 1 |
| Confirmed | 1 |

## 9.6 CRUD Operations

**Q33. Create – add a new category (INSERT)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  INSERT INTO CATEGORY
4    VALUES (105, 'Home Appliances', 'Home Appliance Products');
```

**Q34. Read – view the categories (SELECT)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  SELECT *
4    FROM CATEGORY;
```

Result Grid | Filter Rows: | Edit:

| CATEGORY_ID | CATEGORY_NAME | Description |
|---|---|---|
| 101 | Electronics | Electronic Products |
| 102 | Clothing | Fashion Products |
| 103 | Books | Educational Books |
| 105 | Home Appliances | Home Appliance Products |
| NULL | NULL | NULL |

**Q35. Update – modify a category name (UPDATE)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  UPDATE CATEGORY
4    SET CATEGORY_NAME = 'Home Appliances'
5    WHERE CATEGORY_ID = 105;
```

**Q36. Delete – remove a category (DELETE)**

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  DELETE FROM CATEGORY
4    WHERE CATEGORY_ID = 105;
```

## 9.7 View

### Q37. Create and display the Order_Details view (CREATE VIEW)

```
SHOW DATABASES;
USE COLLEGEDB;
CREATE VIEW Order_Details AS
SELECT
    C.CUSTOMERS_NAME,
    O.ORDER_ID,
    O.ORDER_DATE,
    O.TOTAL_AMOUNT,
    O.ORDER_STATUS
FROM CUSTOMERS C
JOIN `ORDER` O
ON C.CUSTOMERS_ID = O.CUSTOMERS_ID;
```

```
SHOW DATABASES;
USE COLLEGEDB;
SELECT *
FROM Order_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| CUSTOMERS_NAME | ORDER_ID | ORDER_DATE | TOTAL_AMOUNT | ORDER_STATUS |
|---|---|---|---|---|
| Ravi | 301 | 2026-09-28 | 1500.00 | Shipped |
| Anu | 302 | 2026-09-28 | 2500.00 | Pending |
| Kiran | 303 | 2026-09-29 | 800.00 | Delivered |
| Sneha | 304 | 2026-09-29 | 600.00 | Confirmed |

## 9.8 Data Integrity and Validation

### Q38. Duplicate Primary Key test

```
INSERT INTO CUSTOMER
VALUES (1, 'Test', 'test@gmail.com', '9999999999', 'Hyderabad');
```

*Expected result: MySQL rejects the statement with Error 1062 (duplicate entry for key PRIMARY), because Customer_ID 1 already exists.*

***Output:***

*[Insert screenshot of the duplicate-key error here]*

### Q39. Invalid Foreign Key test

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  INSERT INTO PRODUCT
4    VALUES
5    (205, 'Keyboard', 1200.00, 10, 999);
```

*Expected result: MySQL rejects the statement with Error 1452 (a foreign key constraint fails), because Category_ID 999 does not exist in CATEGORY.*

### Q40. CHECK constraint test

```
1 •  SHOW DATABASES;
2 •  USE COLLEGEDB;
3 •  INSERT INTO PRODUCT
4    VALUES
5    (206, 'Mouse', -500.00, 10, 101);
```

*Expected result: MySQL rejects the statement with Error 3819 (a CHECK constraint is violated), because the price must be greater than zero.*

# 10. CONCLUSION & FUTURE ENHANCEMENTS

## 10.1 Conclusion

The implemented database follows the design developed for the E-Commerce Order Management System. The seven related tables organize customers, categories, products, orders, order items, payments, and deliveries. Constraints and normalization up to 3NF support data integrity and reduce redundancy. The SQL operations demonstrate creation, modification, retrieval, CRUD operations, joins, aggregation, and views. The database provides a foundation for efficient order, payment, and delivery management.

## 10.2 Future Enhancements

- Role-based access for administrators, customers, and authorized users.
- Product inventory and stock management.
- Online payment gateway integration and payment tracking.
- Real-time order and delivery tracking.
- Automated sales reports and analytics dashboards.
- Backup, recovery, audit logging, and stronger database security.
- Web or mobile application interface for customers and administrators.

## 11. REFERENCES

Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.

Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

MySQL 8.0 Reference Manual, Oracle Corporation.

Course lecture notes, laboratory exercises, and approved DBMS learning resources.